

Reviewer Pack — Multiplicity-Doubling Lower Bound from Protocol Pullback on ...

Entry 6a6a970e1f5b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 08:37:28 UTC

Multiplicity-Doubling Lower Bound from Protocol Pullback on Indexing Gadgets

Entry ID: 6a6a970e1f5b **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 08:37:28 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Lifting (CGL) / Coarse Geometry & Roe-Algebra methods in Communication Complexity **Field B** (complexity object): Deterministic two-party communication complexity $CC(f \circ G^n)$ of lifted boolean functions

Statement:

Let $G = \text{IND}_b$ be the standard indexing MetricGadget on $X = \{0,1\}^b \times [b]$, $Y = [b]$ with d the Hamming metric on $X \times Y$, and let $f: \{0,1\}^n \rightarrow \{0,1\}$ have deterministic decision-tree complexity $Q(f)$. Then for the lifted input space $((X \times Y)^n, d_\oplus)$ at scale $R = \text{diam}(G)$, every deterministic protocol Π computing $f \circ G^n$ satisfies $\text{CoarsePullbackProtocol}(\Pi) = (m_\Pi, R_\Pi)$ with $m_\Pi \geq 2^{\{Q(f)\} / \text{poly}(b, n)}$, so by axiom A2, $CC(f \circ G^n) \geq Q(f) - O(\log(bn))$. Equivalently, $\text{CLC}(f, \text{IND}_b) \geq Q(f) \cdot \log_2(\text{asdim}_R(\text{IND}_b^{\otimes n}) + 1) - O(\log(bn))$, recovering the Raz-McKenzie bound through the coarse-cover route.

Rationale (proposer's reasoning):

This sub-conjecture directly tests axiom A2 (Protocol $\rightarrow$ Cover) on the canonical gadget for which Raz-McKenzie lifting is known to hold, using axiom A3 (Asdim Amplification) to guarantee that the iterated indexing gadget has unbounded coarse dimension proportional to n . If A2 is sound, the multiplicity of the protocol-induced RoeCover must scale exponentially in the query complexity, otherwise one could collapse a query-tree lower bound through a low-multiplicity cover — contradicting the structure theorem at the heart of CGL. Confirming the bound on small parameters provides empirical evidence that the coarse-cover viewpoint reproduces the standard query-to-communication lift, a necessary sanity check before pursuing axiom A4's non-relativizing strengthening.

Taxonomy category: LIFTING (status at proposal time:)

Framework membership: framework fw_6997a27304, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c5043c501c5b6d85

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 20 trials (covering AND, OR, PARITY, Tribes_{2,2}, plus random f) over $b \in \{2,3,4\}$, $n \in \{2,3,4\}$, the inequality $CC(f \circ G^n) \geq Q(f) - 2 \cdot \log_2(bn)$ must hold in $\geq 90\%$ of trials, with mean slope of CC vs $Q(f)$ (linear regression) ≥ 0.9 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - deterministic communication complexity lifting indexing gadget Raz McKenzie decision tree - query-to-communication lifting theorem indexing gadget deterministic lower bound - coarse geometry asymptotic dimension communication complexity Roe algebra lifting

Top relevant hits considered: - [<http://arxiv.org/abs/2211.17214v1>] Lifting to Parity Decision Trees Via Stifling - [<http://arxiv.org/abs/2105.01963v3>] One-way communication complexity and non-adaptive decision trees - [<http://arxiv.org/abs/2512.11833v1>] Soft Decision Tree classifier: explainable and extendable PyTorch implementation - [<http://arxiv.org/abs/1904.13056v4>] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [<http://arxiv.org/abs/2012.03415v1>] On Query-to-Communication Lifting for Adversary Bounds - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1903.06084v8>] Lifting coarse homotopies - [<http://arxiv.org/abs/2312.08907v2>] Roe algebras of coarse spaces via coarse geometric modules

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import product, combinations

def hamming_distance(x, y):
    return sum(xi != yi for xi, yi in zip(x, y))

class MetricGadget:
    def __init__(self, X, Y, g):
        self.X = X
        self.Y = Y
        self.g = g
```

```

def distance(self, x1, x2):
    return hamming_distance(x1, x2)

def generate_truth_table(f, domain):
    return {x: f(x) for x in domain}

def partition_space(space, num_partitions):
    partitions = []
    n = len(space)
    step = n // num_partitions
    for i in range(num_partitions):
        start = i * step
        end = (i + 1) * step if i < num_partitions - 1 else n
        partitions.append(space[start:end])
    return partitions

def find_min_cost_protocol(truth_table, max_depth=100):
    def dfs(node, depth):
        if depth == max_depth:
            return float('inf'), []
        min_cost = float('inf')
        best_partition = None
        for partition in partitions:
            cost = 0
            for x in node:
                y = truth_table[x]
                if not any(hamming_distance(x, p) < radius for p in partition):
                    cost += 1
            if cost < min_cost:
                min_cost = cost
                best_partition = partition
        return min_cost, [best_partition]

    partitions = partition_space(list(truth_table.keys()), 2)
    return dfs({}, 0)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    b = random.choice([2, 3, 4])
    n = random.choice([2, 3, 4])

    X = list(product(range(2), repeat=b)) + list(range(b))
    Y = range(b)
    g = lambda x, y: x[y]
    G = MetricGadget(X, Y, g)

    def AND(x):
        return all(xi == 1 for xi in x)

    def OR(x):
        return any(xi == 1 for xi in x)

    def PARITY(x):
        return sum(xi for xi in x) % 2

    def Tribes(k, n):

```

```

    if len(x) != k * n:
        raise ValueError("Input length must be k*n")
    return all(sum(x[i:i+n]) % 2 == 1 for i in range(0, len(x), n))

functions = [AND, OR, PARITY]
if random.random() < 0.5:
    functions.append(lambda x: Tribes(2, 2)(x))

results = []
for f in functions:
    truth_table = generate_truth_table(f, X)
    protocol_cost, _ = find_min_cost_protocol(truth_table)
    R = max(hamming_distance(x, y) for x in X for y in Y)
    m_Pi = 2 ** protocol_cost / (b * n)
    CC_f_Gn = protocol_cost
    Q_f = len(truth_table)
    if CC_f_Gn >= Q_f - 2 * math.log2(b * n):
        results.append((True, ""))
    else:
        results.append((False, f"CC(f∘G^n)={CC_f_Gn} < {Q_f} - 2*log2({b}*{n})"))

conjecture_holds = all(result[0] for result in results)
counterexample = "; ".join(result[1] for result in results if not result[0])
return {
    "metric_name": "CC(f∘G^n)",
    "metric_value": sum(CC_f_Gn for CC_f_Gn, _ in results) / len(results),
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) if x else 11 for x in input().split()]
    if not seeds:
        seeds = [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_38168056.py", line 114, in <module>
    seeds = [int(x) if x else 11 for x in input().split()]
                ^^^^^^^
```

EOFError: EOF when reading a line

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an EOFError while reading seeds from stdin before any data could be produced, so neither the support nor falsification conditions can be evaluated. | next: Patch the harness to provide default seeds (or read from argv/env) when stdin is empty, then rerun the ≥ 20 -trial battery over $b \in \{2,3,4\}$, $n \in \{2,3,4\}$.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	5637
2	novelty	claude_max	opus	0	0	3578
3	novelty	claude_max	opus	0	0	12255
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14444
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12819
6	judge	claude_max	opus	0	0	6324

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 55057 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/6a6a970e1f5b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6a6a970e1f5b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6a6a970e1f5b.tar.gz` (if generated)